

Preguntas y respuestas — Demo Harness

1. ¿Debo borrar todo /src menos el README?

En esta copia local ya está así — /src solo tiene README.md, nada más. Así tiene que estar **antes** de arrancar la demo, para que la audiencia vea el rojo real y después vea a Copilot construirlo en vivo. Si en la PC del trabajo ya probaste el flujo y quedó algo implementado ahí, sí: borra todo dentro de src/ excepto README.md antes de presentar (o hacé `git checkout HEAD -- src && git clean -fd src` si esa carpeta también es un repo git).

2. ¿La demo en vivo es correr simplemente el texto que me diste en Copilot?

No es *solo* eso — son 3 acciones en vivo, en este orden:

- (a) Correr `verify.ps1` con `/src` vacío → mostrar el rojo.
- (b) Pegar el prompt en Copilot (modo Agent) → dejarlo iterar hasta verde.
- (c) Inyectar la violación a mano, correr `verify.ps1` de nuevo → mostrar que la caza.

El prompt a Copilot es la parte (b), la más larga, pero no es toda la demo.

3. ¿Qué es feedforward?

Es todo lo que le das al agente **antes** de que escriba una línea de código, para guiarlo. En este caso: `specs/constitution.md` y `specs/products-spec.md`. Es "hacia adelante" porque actúa antes del hecho, previniendo — le decís las reglas de antemano en vez de corregir después. Se complementa con el **feedback**, que es lo que verifica *después* (los gates). Harness = feedforward + feedback trabajando juntos.

4. ¿Qué hacen `verify.ps1` y `verify.sh`?

Son el mismo script, escrito dos veces para dos sistemas operativos (`.ps1` = Windows/PowerShell, `.sh` = Linux/Mac/bash) — se usa `verify.ps1` en Windows. Ambos hacen lo mismo:

1. Compilan `/gates`.
2. Si `src/Api/Api.csproj` ya existe, levantan la API en `localhost:5087`.
3. Corren los 18 tests contra esa API.
4. Apagan la API.
5. Devuelven exit code 0 (verde) o distinto de 0 (rojo).

5. ¿Qué hacen los gates? ¿Son el harness?

Los gates son los chequeos automáticos: 13 tests que le pegan por HTTP a la API para validar comportamiento, y 5 reglas que inspeccionan el código compilado para validar que se respeta la arquitectura en capas. Deciden si el código "entra" o no. **Son el feedback, no todo el harness.** El harness completo = specs (feedforward) + gates (feedback) + el script que los conecta (el loop). Los gates son "el corazón" en el sentido de que son la parte escrita con más cuidado, pero sin las specs no tendrían qué hacer cumplir.

6. Paso C — ¿cómo inyecto la violación exactamente?

Abrir `src/Api/Controllers/ProductsController.cs` (va a existir recién después del paso B, cuando Copilot ya implementó todo) y agregar esto:

```
```csharp using Microsoft.EntityFrameworkCore; // <- agregar este using arriba

public sealed class ProductsController : ControllerBase { private readonly
IProductService _productService; private readonly DbContext _db = null!;
// <- agregar este campo ... ```
```

Si al compilar tira error porque `Api.csproj` no tiene el paquete de EF Core, agregar esta línea dentro de un `<ItemGroup>` en `src/Api/Api.csproj`:

```
xml <PackageReference Include="Microsoft.EntityFrameworkCore"
Version="8.0.10" />
```

Después correr `.\scripts\verify.ps1` de nuevo.

### Qué se va a ver fallar: el test

`ARCH01_Api_DoesNotDependOn_EntityFrameworkCore`, con un mensaje que dice `Failing types: Api.Controllers.ProductsController` — señala el nombre exacto de la clase culpable. El resto (17 de 18) va a seguir en verde.

Después borrar esas dos líneas agregadas (y la del `csproj` si se agregó) y correr `verify.ps1` una vez más para cerrar en verde de nuevo.